Docker: image, container, volume và network

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Docker: image, container, volume và network

Image là gói chỉ đọc chứa filesystem và metadata cần để tạo container. Container là một tiến trình được chạy từ image trong môi trường cô lập tương đối. Một image có thể sinh ra nhiều container. Tag là tên thuận tiện nhưng có thể thay đổi trỏ tới image khác; digest định danh nội dung cụ thể hơn. Khi cần tái tạo release, hãy ghi tag/version và digest hoặc cơ chế định danh tương đương.

Dockerfile là chỉ dẫn build image theo các layer. Lớp thay đổi ít nên đặt trước để cache hữu ích; phụ thuộc nên được khóa phiên bản phù hợp. Không đưa secret vào Dockerfile, biến build hoặc layer. Một image nhỏ, chạy bằng user không đặc quyền và chỉ chứa thứ cần thiết giúp giảm bề mặt tấn công, nhưng quan trọng nhất là hiểu dependency và quy trình vá lỗi.

Filesystem bên trong container có thể mất khi container được thay thế. Dữ liệu cần tồn tại phải nằm ở volume, dịch vụ database hoặc storage phù hợp. Nếu hai container cần nói chuyện, chúng cần địa chỉ và network thích hợp; **localhost** bên trong container API là chính container API, không phải container database. Trong Compose, các dịch vụ cùng network có thể được gọi bằng tên dịch vụ theo cấu hình Compose.

Lab Docker từng bước

Điều kiện: Docker đã cài và daemon chạy. Trước tiên thử **docker version**; nếu lỗi, ghi thông báo và kiểm tra Docker Desktop/daemon. Tạo Dockerfile cho ứng dụng mẫu, build với tag riêng như **notes-api:lab1**, rồi chạy container gắn port local. Dùng **docker ps** xác nhận tiến trình, **docker logs <ten>** để xem log và trình duyệt hoặc **curl** gọi health endpoint. Dừng và xóa container thử nghiệm khi xong; image có thể giữ lại để so sánh.

Khi thêm database qua Compose, khởi động stack, tạo một ghi chú, restart container API và xác minh ghi chú còn tồn tại. Sau đó giải thích data nằm ở đâu. Cố ý đặt host database là **localhost** và quan sát lỗi; sửa về tên dịch vụ. Không xóa volume chứa dữ liệu quan trọng; mọi thao tác xóa chỉ trong lab dùng dữ liệu giả.

Lỗi thường gặp

"Container running" chỉ nói tiến trình chính còn sống, không chứng minh ứng dụng dùng được. Port trong container chưa tự động mở cho máy ngoài; cần mapping hoặc network phù hợp. Docker build thành công không có nghĩa image an toàn hay test đã qua. Health check chỉ có ích khi kiểm tra được khả năng mà hệ thống phụ thuộc, với thời gian và tần suất hợp lý.

Phân tích Dockerfile và Compose thực tế

Trong **lesson/labs/notes-api/Dockerfile**, dòng **FROM** chọn image nền có Python; **WORKDIR** đặt thư mục làm việc; **COPY** mang code vào image; **RUN** tạo thư mục dữ liệu; **USER** chuyển sang người dùng không đặc quyền; **ENV** đặt cấu hình mặc định; **EXPOSE** ghi lại cổng ứng dụng; **CMD** chỉ định tiến trình khởi chạy. **EXPOSE** không tự mở cổng ra máy bạn: bước **docker run -p** hoặc Compose **ports** mới ánh xạ cổng. Trong lab, server phải nghe **0.0.0.0** bên trong container để lưu lượng từ bên ngoài container tới được; khi chạy trực tiếp trên máy, mặc định chỉ nghe **127.0.0.1** để an toàn hơn.

Compose mô tả cách chạy các dịch vụ local bằng file. Ở bản lab này chỉ có một service API và một named volume. Volume gắn với **/data**; SQLite ghi file vào đó. Mapping **127.0.0.1:8000:8000** chỉ mở ở loopback của máy chủ Docker, giảm nguy cơ vô tình công khai lab ra mạng. Khi chuyển sang cloud hoặc cluster, cấu hình truy cập sẽ khác và cần thiết kế riêng.

Lab tái lập, có kết quả mong đợi

Điều kiện: Docker Engine/Desktop đang chạy, cổng 8000 chưa bị chiếm, thư mục lab không chứa dữ liệu quan trọng. Mở terminal tại **lesson/labs/notes-api**, chạy **docker compose build** rồi **docker compose up -d**. Dùng **docker compose ps** để thấy service đang chạy. Gọi **http://127.0.0.1:8000/health**; mong đợi JSON **status: ok**. Gửi POST tạo một ghi chú như ở Giai đoạn 1, rồi gọi GET để xác nhận.

Chạy **docker compose restart api**; GET lại. Ghi chú phải còn vì database nằm trong named volume. Tiếp tục chạy **docker compose down** rồi **docker compose up -d**; GET lại. Dữ liệu vẫn còn nếu bạn **không** dùng tùy chọn xóa volume. Kết thúc lab bằng **docker compose down**. Chỉ khi bạn muốn xóa dữ liệu lab giả, kiểm tra tên project/volume rồi dùng tùy chọn xóa volume theo tài liệu Docker. Đừng dùng nó khi volume có dữ liệu cần giữ.

Nếu health không mở được, điều tra theo thứ tự: **docker compose ps** có service không; **docker compose logs api** có lỗi không; port 8000 bị chiếm không; có nhập đúng **127.0.0.1** không. Nếu ứng dụng chạy nhưng trả 500 sau khi ghi, xem quyền **/data** và log database. Không sửa Dockerfile, Compose và code cùng lúc; thay một yếu tố rồi thử lại.

Câu hỏi tư duy

Vì sao **docker compose down** không nên được xem là backup? Vì sao đổi code rồi **docker compose restart** chưa chắc chạy code mới? Vì sao image đã build không chứa ghi chú vừa tạo khi volume được gắn? Đáp án gợi ý: volume là dữ liệu riêng, không phải bản sao phục hồi; restart dùng container/image hiện có, cần build và recreate để nhận mã mới; dữ liệu phát sinh được ghi vào volume ngoài image.

Từ Dockerfile đến tiến trình đang chạy

Dockerfile là công thức build image; image là các lớp nội dung và metadata; container là một tiến trình (hoặc nhóm tiến trình) chạy từ image với filesystem writable riêng, network namespace và giới hạn tài nguyên theo cấu hình. Dừng container không đồng nghĩa xóa image; xóa container có thể xóa dữ liệu ở writable layer. Volume/bind mount là cách đưa dữ liệu ra ngoài vòng đời container, nhưng volume vẫn cần backup. Nếu app dùng SQLite ở **/data/notes.db**, chạy image mà không mount **/data** có thể làm ghi chú biến mất khi thay container.

Quy trình đọc Dockerfile: xác định base image và phiên bản, thứ tự **COPY**/**RUN**, user cuối cùng, cổng, biến môi trường, đường ghi, lệnh khởi động. Base image nhỏ không mặc nhiên an toàn; vẫn cần update/quét dependency và kiểm tra bản vá. Dùng tag cố định hoặc digest cho build tái tạo được. **EXPOSE** chỉ là metadata mô tả cổng; publish cổng từ host là lựa chọn riêng khi **docker run -p** hoặc Compose **ports**.

Lab 1: image bất biến, dữ liệu tồn tại

Tại **lesson/labs/notes-api**, đọc Dockerfile rồi build **docker build -t notes-api:lab-1 .**. Chạy container lab với **-p 127.0.0.1:8000:8000** và volume gắn **/data**. Gọi **GET /health**, tạo ghi chú, đọc lại. Dừng/xóa container, chạy container mới từ **cùng image** với **cùng volume**, xác nhận ghi chú còn. Lặp lại với volume mới để thấy dữ liệu không tự đi theo image. Ghi tên volume, ID image, ID container và nội dung DB; đó là bằng chứng phân biệt các khái niệm. Khi dọn dẹp, chỉ xóa container/volume lab đã xác nhận tên, không dùng lệnh prune toàn máy nếu có dự án khác.

Nếu chạy Compose, **docker compose up --build -d**, xem **docker compose ps**, **docker compose logs**, rồi **docker compose down**. Đừng thêm **-v** nếu bạn còn cần volume dữ liệu lab. Trong YAML Compose, **build** tạo image, **environment** đặt cấu hình runtime, **volumes** nối dữ liệu, **ports** xuất cổng ra host. Container trong cùng network gọi nhau bằng tên service và container port; **localhost** trong container chỉ trỏ về chính container đó.

Lab 2: ba lỗi có chủ đích

- Đổi host port 8000 sang 8001 nhưng vẫn gọi 8000. Kiểm tra **docker compose ps** và mapping; không sửa app khi lỗi ở cổng host.
- Đổi biến **NOTES_DB** sang đường không ghi được dưới non-root user. **GET /health** có thể vẫn 200 nhưng **POST /notes** lỗi; xem log và quyền filesystem. Sửa về **/data/notes.db** đã mount writable.
- Đổi tên service DNS trong cấu hình reverse proxy hoặc app phụ thuộc. Kiểm tra container network và tên service. Đừng dùng IP container cố định vì có thể đổi khi tái tạo.

Sau mỗi lỗi, ghi giả thuyết, lệnh kiểm tra, output then chốt, sửa một biến và xác nhận. Cách này quan trọng hơn việc thuộc lòng toàn bộ option Docker.

Bảo mật, tài nguyên và vận hành

Chạy non-root giảm tác động nếu app bị khai thác, nhưng không thay việc kiểm soát secret, filesystem, network và image. Không **COPY .** bừa nếu thư mục chứa **.env**, private key hoặc test data; dùng **.dockerignore**. Không đưa secret vào **ARG**/**ENV** khi build vì có thể lộ qua layer/history hoặc registry. Đặt memory/CPU limit trong môi trường phù hợp và xem OOM exit code khi app restart. Log nên ra stdout/stderr để collector thu; file log bên trong container dễ mất khi thay container.

Container không phải VM: nó dùng kernel host; isolation có giới hạn và phụ thuộc cấu hình runtime. Một container **healthy** theo healthcheck đơn giản cũng chưa chứng minh chức năng ghi DB thành công. Cần kết hợp smoke test và telemetry khi release.

Image, container và host: ba ranh giới phải phân biệt

Image là tập lớp filesystem và metadata bất biến theo digest; container là tiến trình được chạy từ image với lớp ghi riêng, cấu hình, namespace mạng/process và giới hạn tài nguyên. Docker Engine chạy trên host; container dùng kernel của host chứ không có một kernel riêng như VM. Vì vậy image Linux không tự biến Windows host thành Linux production giống hệt; Docker Desktop thường dùng VM/WSL bên dưới. Container giúp đóng gói phụ thuộc ứng dụng, nhưng không tự tạo HA, backup hay bảo mật đầy đủ.

Build context là tập file gửi cho builder. Nếu copy cả **.git**, **.env** hoặc DB vào context, chúng có thể lọt vào image/layer hoặc làm build chậm. **.dockerignore** giới hạn context; secret build cần cơ chế secret mount phù hợp, không dùng **ARG PASSWORD** rồi hy vọng layer xóa được. Dockerfile **COPY requirements.txt** trước **COPY .** giúp cache bước cài dependency khi chỉ source đổi. Cache chỉ ảnh hưởng tốc độ build; muốn reproducible cần khóa dependency và pin base image/digest theo quy trình cập nhật bảo mật. Tag **notes:latest** là tên có thể di chuyển; digest định danh nội dung image.

Port và filesystem qua hai ví dụ

App trong container nghe **0.0.0.0:8000**. **docker run -p 8080:8000** công bố cổng host 8080 tới cổng container 8000; **EXPOSE 8000** trong Dockerfile chỉ là metadata, không tự publish. Nếu app chỉ bind **127.0.0.1 bên trong container**, host publish port vẫn có thể không tới process theo cách bạn mong; sửa bind address của app, không sửa ngẫu nhiên **-p**.

Writable layer của container mất khi container bị xóa; named volume có vòng đời riêng; bind mount nối trực tiếp đường host vào container. Với SQLite, nếu DB nằm trong layer, xóa/tạo container mới có thể làm mất dữ liệu. Volume không phải backup: bạn vẫn có thể xóa dữ liệu trong volume, làm hỏng file hoặc xóa cả volume. Backup cần bản sao độc lập và restore test. Bind mount tiện dev nhưng phụ thuộc đường host/quyền, dễ khác giữa máy người học và CI.

Lab đọc một image thay vì chỉ chạy lệnh

Điều kiện: Docker đang chạy, dùng dự án **lesson/labs/notes-api**. Trước build, đọc Dockerfile và ghi: base image, workdir, các **COPY**, dependency, user chạy và lệnh khởi động. Chạy **docker build -t notes-lab:study .** trong thư mục lab. Quan sát output lần đầu và lần thứ hai; bước nào dùng cache? Dùng **docker image inspect notes-lab:study** xem config, **docker history notes-lab:study** xem lớp. **Kết quả dự kiến:** có image tag local; digest/ID và cache behavior thay đổi theo builder/máy. Đừng chép ID mẫu vào ghi chú.

Chạy container theo hướng dẫn app, map cổng host, rồi kiểm tra **docker ps**, **docker logs**, **docker inspect** và gọi API. Ghi chính xác ba địa chỉ: cổng process trong container, cổng publish trên host và URL client gọi. Gây lỗi bằng cách chạy một container thứ hai cùng host port: Docker báo port đã dùng. Đổi **host port** của container thứ hai, giữ container port nếu app vẫn nghe 8000. Giải thích vì sao sửa **EXPOSE** không giải quyết port conflict ở host.

Tạo dữ liệu thử trong DB của app, xóa **container lab** rồi tạo lại không gắn volume để quan sát dữ liệu mất; lặp lại với named volume đúng đường DB để thấy dữ liệu còn. Làm trong lab riêng vì thao tác xóa container/volume phá dữ liệu. Dọn bằng cách dừng/xóa đúng container và volume có tên lab sau khi đã

ghi bằng chứng, kiểm tra tên bằng **docker ps -a** và **docker volume ls** trước khi xóa; không chạy lệnh prune toàn máy.

Mạng Compose và dependency readiness

Compose tạo mạng cho service; trong cùng mạng, app gọi DB bằng tên service như **db:5432**, không dùng **localhost:5432** vì localhost của app container là chính app container. **depends_on** giúp thứ tự tạo/chạy theo cấu hình, nhưng app vẫn phải chịu được DB khởi động chậm hoặc mất kết nối sau đó. Healthcheck có thể giúp điều phối readiness ban đầu, không thay retry có backoff ở ứng dụng. Với nhiều service, tách lỗi "DNS không phân giải", "TCP không connect", "auth bị từ chối" và "schema chưa có". Mỗi lỗi dẫn tới cách sửa khác.

Nếu container thoát ngay, xem **docker ps -a** và **docker logs <ten>**; exit code 0 có thể do command kết thúc bình thường nhưng bạn mong web server chạy mãi. **OOMKilled** liên quan memory; **permission denied** có thể do image user không đọc được bind mount; **no such file** có thể vì Dockerfile copy thiếu file hoặc workdir sai. Restart policy chỉ lặp lại lỗi nếu cấu hình sai. **docker exec** hữu ích để quan sát nhưng sửa file trực tiếp trong container không tồn tại sau recreate; sửa source/Dockerfile/config rồi rebuild/redeploy.

Từ lab tới production

Chạy process dưới user không phải root nếu image/app cho phép; giới hạn capability, read-only filesystem khi phù hợp, scan image và cập nhật base image có kiểm soát. Không đưa secret vào env/log/image nếu có lựa chọn quản lý secret tốt hơn; biến môi trường dễ bị lộ qua cấu hình/quyền inspect. Thiết kế readiness/liveness ở orchestration theo hành vi app; Docker một mình không cân tải hay thay node hỏng. Với registry private như Harbor, chứng minh build agent đẩy được image, **node triển khai** kéo được image qua DNS/TLS/credential, và digest ở production đúng digest được phê duyệt.

Ca có lời giải: local chạy, CI build thành công, Pod production **ImagePullBackOff**. Trước hết đọc Events của Pod: **not found** khác **unauthorized**, TLS x509 hoặc timeout. So full image reference/digest, registry credential của namespace, DNS/TLS từ node và quyền project Harbor. Không sửa Dockerfile nếu image đã tồn tại đúng digest mà node không thể pull. Nguồn: Docker build cache, Docker volumes, Docker bind mounts.

Bài đọc Dockerfile theo từng lớp

Với Dockerfile của Notes API, đánh dấu dòng nào ảnh hưởng cache khi **requirements.txt** đổi và khi chỉ **app.py** đổi. **COPY . .** sớm sẽ làm bước cài package sau nó mất cache mỗi lần source đổi. Đưa file dependency trước, cài, rồi copy source giúp build nhanh hơn, nhưng phải bảo đảm dependency được khóa và **.dockerignore** loại file nhạy cảm. Multi-stage build có thể tách tool build khỏi runtime image; image nhỏ hơn giảm bề mặt và thời gian pull, nhưng vẫn cần scan và cập nhật base. Một image chứa **curl**/shell có thể dễ debug, nhưng production image tối giản có tradeoff điều tra; dùng ephemeral debug container/telemetry theo chính sách thay vì để mọi công cụ trong image.

Container restart và dữ liệu

docker restart giữ cùng container và writable layer, còn **docker rm** rồi **docker run** tạo container khác, writable layer cũ không còn. Nếu bạn test persistence chỉ bằng **restart**, bạn chưa chứng minh dữ liệu sống qua recreate/deploy. Thử với named volume và kiểm tra volume path đúng đường app ghi. Một volume vẫn có thể bị hỏng/xóa; backup cần bản sao ra ngoài Docker host, policy retention và restore test. Nếu bind mount từ Windows vào Linux VM của Docker Desktop, quyền/đường dẫn/performance có thể khác server Linux; ghi môi trường lab để tránh kết luận quá rộng.

Exit code và signal

Container chính sống theo PID 1; nếu command khởi động shell rồi shell thoát hoặc không chuyển signal đúng, shutdown có thể không graceful. Khi orchestrator gửi SIGTERM và chờ grace period, app cần ngừng nhận request, hoàn tất công việc trong giới hạn rồi thoát. Nếu process ignore signal, bị SIGKILL, có thể mất request hoặc corrupt dữ liệu nếu app không xử lý tốt. Dockerfile **CMD** dạng exec thường giúp signal tới process trực tiếp hơn dạng shell; kiểm tra entrypoint thực. **OOMKilled** khác exit do exception: xem trạng thái, limit và log trước khi tăng memory.

Tình huống phỏng vấn có lời giải

Docker build xanh, container thoát ngay với code 0. Có thể command chạy tác vụ ngắn rồi kết thúc; kiểm tra **CMD**/entrypoint, process có web server blocking không. **App gọi localhost:5432 nhưng DB ở Compose service db.** Localhost trong app container là app container; dùng tên service/mạng và port DB nội bộ. **Host gọi localhost:8080 bị refused nhưng docker ps Up.** Kiểm tra publish port, app bind address, process cổng trong container và firewall host. **Docker scan báo CVE trong base image.** Xác định version/đường khai thác/patch, cập nhật base có test; không xóa package tùy tiện làm app hỏng.

Nguồn: Dockerfile best practices, Docker container networking, Docker volumes.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://docs.docker.com/get-started/docker-concepts/
- https://docs.docker.com/compose/

Phụ lục: mã lab để đọc offline

lesson/labs/notes-api/Dockerfile

```
FROM python:3.13-slim
WORKDIR /app
COPY app.py /app/app.py
RUN mkdir /data && chown 10001:10001 /data
USER 10001:10001
ENV NOTES_HOST=0.0.0.0 NOTES_PORT=8000 NOTES_DB=/data/notes.db
EXPOSE 8000
CMD ["python", "/app/app.py"]
```

lesson/labs/notes-api/compose.yaml

```
services:
  api:
    build: .
    image: notes-api:lab1
    ports:
      - "127.0.0.1:8000:8000"
    volumes:
      - notes_data:/data
volumes:
  notes_data:
```